TERRAMON

NEURAL NETWORK GAME

Architecture Document

AI Engineering from Scratch — 20 Phases Applied

Rohit Ghumare → Terramon TMA · July 2026

503 lessons · 25 files · 3,613 lines changed · 84 tests

Every algorithm built from raw math before a single framework gets imported.

1. Executive Summary

Terramon is a Telegram Mini App where players summon AI creature agents from their thoughts. Every creature is a real AI agent — not a label or hardcoded NPC. The game implements the full AI engineering stack from scratch: an autograd engine with backpropagation, a Mixture of Experts network with 12 Jungian archetype experts, a Bayesian belief router, a hashed-feature embedding classifier, and an LLM behavior system with archetype-specific voices.

This document describes the neural network architecture underlying Terramon, developed by applying all 20 phases of the AI Engineering from Scratch curriculum (503 lessons). The result: 25 source files, 3,613 lines of AI/ML code, verified by 84 passing tests.

APPLICATION	TMA UI (Reflex)	Summon, Care, Evolve, Mint, Scout
	Game Loop	REASON -> ACT -> REWARD -> REFLECT -> REPEAT
MODEL	MoE Network (12 experts)	Jungian archetypes + FastRouter
	LLM Behavior	DeepSeek V4 Flash, 12 voices
	Embedding Classifier	TF-IDF + Naive Bayes + KNN
INFRASTRUCTURE	Autograd Engine	Value class, MLP, Adam, BatchNorm
	Memory & Events	JSONL persistence, EventBus, safety middleware

2. Pipeline Overview

Text-to-Creature Pipeline

Stage 1: Text -> Vector. BPE Tokenizer -> Hashing TF-IDF -> 512-dim vector.

Stage 2: Vector -> Archetype. MoE Router -> 12 Jungian expert scores -> archetype.

Stage 3: Archetype -> Creature. Bayesian belief update -> creature stats -> state machine.

Stage 4: Creature -> Response. LLM with archetype voice -> JSON emotion + text.

Key Insight (Chip Huyen)

Most AI engineering problems are in the APPLICATION layer, not the model layer. Terramon's intelligence comes from orchestrating these networks, not from any single model.

3. Autograd Engine

Value Class

Reverse-mode autodiff from scratch. Wraps a scalar, records operations in a computational graph, computes gradients via topological sort backward pass.

Supported ops: add, mul, pow, neg, sub, truediv, relu, tanh, sigmoid, gelu.

Adam Optimizer

Adaptive Moment Estimation with bias correction. Per-parameter moments m and v.

Hyperparameters: lr=0.001, betas=(0.9, 0.999), eps=1e-8.

Weight Initialization (Phase 3)

Xavier init (tanh): W ~ Uniform(-sqrt(6/(fan_in+fan_out)), +sqrt(6/(fan_in+fan_out)))

He init (ReLU): W ~ Normal(0, sqrt(2/fan_in))

BatchNorm1d (Phase 3)

Normalizes activations across batch. Learnable gamma (scale) and beta (shift). Running EMA statistics during eval.

CosineAnnealingLR (Phase 3)

eta_t = eta_min + 0.5*(eta_max - eta_min)*(1 + cos(pi*t/T_max))

4. Mixture of Experts

Architecture

Input: 64-dim encoded vector -> Projection (64->64) -> LayerNorm -> 12 Experts -> Router -> Softmax

Each FastExpert: 2-layer MLP (64->8 tanh + skip -> 1 tanh). 12 experts = 6,348 weights.

Top-K Sparse Routing (Phase 18)

Instead of dense routing to all 12 experts, select top 2-3. The Mixtral 8x7B pattern applied to 12 experts.

~4x faster than dense, prevents expert collapse.

Auxiliary load balancing loss: KL divergence between routing distribution and uniform.

Thinking Loop (Phase 18)

Run N forward passes, accumulate expert scores, take mean. More steps = more robust.

LayerNorm & Dropout (Phase 3)

Pre-expert LayerNorm for training stability. Dropout p=0.1, inverted scaling at inference.

Quantization Demo

MXFP4 (4-bit) packing: 16x compression vs float64, 8x vs float32.

5. Embedding Classifier

Pipeline

Preprocess (NFKC, strip URLs, collapse repeats) -> Tokenize (BPE unigrams+bigrams+trigrams) ->

TF-IDF encode (sublinear TF x smooth IDF) -> L2 normalize -> Cosine to 12 archetype centroids ->

Classify (argmax > 0.05) -> Naive Bayes fallback (Phase 2)

BPE Tokenizer (Phase 5)

Learns ~50 merge rules from 60 archetype example texts. Handles rare words and misspellings via subword decomposition.

TF-IDF++ (Phase 2+5)

Sublinear TF: 1 + log(tf). Smooth IDF: log(N/df) + 1. 512-dim hashed feature space.

Naive Bayes Fallback (Phase 2)

When cosine similarity < 0.05 for ALL archetypes, run per-word likelihood model as second pass.

6. Bayesian Router

Bayesian Inference

P(archetype | thought) proporitonal P(thought | archetype) x P(archetype)

likelihood = cosine(encode(thought), centroid) | prior = Dirichlet([1,1,...1]) | posterior = prior x likelihood

Winner = argmax(posterior). Confidence = max(posterior). Revenue gate at 50% confidence.

Player Insight

Terramon IS the neural network. Creatures ARE the MoE experts. Players train the model by summoning. They earn by MINTing their embedding vector.

7. Creature Agent AI

CreatureState Machine (Phase 6)

7 states: HAPPY, CONTENT, HUNGRY, TIRED, DISTRESSED, SICK, EVOLVING. Each with different decay rates and behavior triggers.

EMA Stat Decay

stat(t) = stat(t-1) * 0.97 (3% loss per tick). Day/night modifiers from time_tool.

Max delta per tick: 15 (gradient clipping concept).

Mood System (Phase 6)

cheerful (avg > 70) -> encouraging messages + bonus XP

content (40-70) -> neutral messages

distressed (< 40) -> urgent messages

Evolution Probability

z = (level-10)/3 + (happiness-70)/10 + (total_xp-500)/200

P(evolve) = sigmoid(z) = 1 / (1 + exp(-z))

8. LLM Behavior System

12 Archetype Voices (Phase 8)

Each Jungian archetype gets a unique voice in the LLM prompt:

Sage=riddles, Jester=humor, Rebel=defiance, Lover=warmth, etc.

Structured Context (Phase 7)

6 attention channels: Identity, State, History, Memory, INSIGHT, Geo.

Chain-of-Thought Emotion (Phase 8)

LLM returns JSON: {emotion, message}. CoT prefix stripped before display.

Per-interaction temperature: talk=0.9, evolve=0.7, summon=0.85, tick=0.75.

Response length decays: 200 tokens (first 2) -> 100 tokens (after 5).

Graceful Degradation

Exponential backoff retry (2 attempts). If all fail, template fallback.

9. Training Loop

MoE Training

Optimizer: Adam (lr=0.001) + CosineAnnealingLR (T_max=100)

Loss: cross-entropy + auxiliary load balancing KL loss

Weight init: Xavier, BatchNorm: learnable gamma/beta

Data: 768 train / 192 test / 12 classes

Gradient verification: central difference, |analytical-numerical| < 1e-6

10. Phase Summary — 20 Phases Applied

All 20 phases applied to Terramon. 25 files, +3,613/-502 lines, 84 tests.

Ph	Focus	Key Changes
0	Setup & Tooling	Docker multi-stage, CI, env validation, data management, error handling
1	Math Foundations	math_utils, Adam, GELU, gradient check, KL divergence
2	Classical ML	IDF weighting, trigrams, NaiveBayes, KNN, weighted keywords, precision/recall
3	Neural Networks	Xavier/He init, BatchNorm1d, CosineAnnealingLR, Dropout, LayerNorm
4	Computer Vision	FAL.ai retry, portrait cache+registry, SVG fallback, image metadata
5	NLP	BPE tokenizer, TF-IDF++, text preprocessing, attention scoring
6	Sequence Models	CreatureState EMA, mood system, day/night cycle, gradient clipping
7	Transformers	Structured LLM context, KV memory, JSON emotion+message parsing
8	LLMs	12 archetype voices, CoT, sampling params, retry, length decay
9	Pretraining	Data versioning, JsonMemory report_stats()
10	Finetuning	LoRA adapter (MoELoRAStack, rank=8, alpha=16)
11	Alignment	EventBus middleware, content safety flagging
12	Agents & Tools	ToolPort, web_search+fetch, Scout agent integration
13	Multimodal	Portrait cache key, pipeline docstring
14	Inference	Railway cold-start, healthcheck endpoint
15	Eval/CI	.coveragerc, CI coverage, badge
16	Safety	Payment rate limiting (3 rare/hr)
17	MLOps	Docker healthcheck, portrait auto-cleanup
18	Advanced	Top-k sparse MoE, thinking loop, long-context insight, reasoning chain
19	Capstone	Frontend: state+mood, portrait, Scout button, BPE info, safety flag, /health

11. Numerical Verification

The following verified properties ensure mathematical correctness across the neural network stack.

Property	Method	Result
Softmax stability	log-sum-exp trick	Sum=1.0, handles large/small/negative inputs
Sigmoid stability	Branch: x>=0 vs x<0	sig(0)=0.5, sig(-100)=0, sig(100)=1
L2 normalization	Euclidean norm	norm=1.0 on all valid text inputs
Cosine similarity	Dot over L2-normed	self=1.0, symmetric, bounded [0,1]
Dirichlet sampling	Log-space alphas	Bounded [0,1], deterministic seed
Evolution sigmoid	Logistic function	Smooth P(evolve), monotonic in stats
Adam optimizer	Bias-corrected moments	m_hat, v_hat converge to unbiased estimates
Gradient check	Central difference	|analytical - numerical| < 1e-6
Cross-entropy	Softmax + negative log	Loss >= 0, zero only at perfect prediction
BatchNorm	Running EMA	Train=batch, eval=running, dimensions preserved
Top-k sparsity	k=2 vs dense	Only k non-zero routing weights
Load balancing	KL(router || uniform)	Zero at perfect balance, positive at skew

Source Map

File	Lines	Contains
autograd.py	+318	Value, MLP, Adam, BatchNorm1d, CosineAnnealingLR
k3_insight_engine.py	+593	FastExpert, MoENetwork, Dropout, LayerNorm, training loop
embedding_classifier.py	+319	TF-IDF, trigrams, BPE, NaiveBayes, KNN
creature_agent.py	+357	CreatureState, EMA decay, mood, evolution sigmoid
llm_behavior.py	+719	12 archetype voices, structured context, CoT, retry
math_utils.py	NEW	logsumexp, softmax, sigmoid, Adam, KL divergence
bpe_tokenizer.py	NEW	BPE learn/tokenize, subword decomposition
text_preprocessing.py	NEW	NFKC, URL strip, emoji strip, stop words
lora_adapter.py	NEW	LoRAConfig, LoRALinear, MoELoRAStack
portrait_gen.py	+98	FAL.ai retry, cache, registry, SVG fallback

84 tests pass. Key test files: test_art_pipeline.py (+350 lines), test_embedding_classifier.py (+106 lines), test_game_loop.py (10 roast failure modes).